딥러닝 기반 저선량 CT 영상 복원 및 잔차 연결·오류 원인 분석

Low-Dose CT Image Reconstruction with U-Net Variants: Original Raw DICOM ($[-100, 300]$ HU) Pipeline, Residual Learning, and Error Source Analysis

교과목: AI for Medical Image Analysis (성균관대학교 메타바이오헬스학과) | **학번·성명:** 2026710517 심순용 | **제출일:** 2026. 11. 08.

[보고서 요약 (Summary)]

• **원본 Raw DICOM 데이터 및 $[-100, 300]$ HU 윈도우 전처리:** 리사이즈된 256×256 PNG 대신 원본 512×512 `.IMA` Raw 데이터(3mm slice thickness, Sharp kernel, 총 10명 2378장 쌍)를 사용하였으며, 시각적 복원 결과를 $[-100, 300]$ HU 감쇠 범위로 역변환하여 제시함. 테스트 환자(L192, L506, 총 451장)는 학습·검증 과정에서 분리하여 평가함.

• **아키텍처·증강 실험 및 잔차 연결(Residual Connection) 비교:** U-Net 계열 네트워크(`U_Net`, `U_Net_NoRes`, `R2U_Net`, `AttU_Net`, `R2AttU_Net`, `NestedUNet`)와 MONAI 기반 데이터 증강 조합을 실험함. 입력을 출력에 더해주는 잔차 연결 유무(`U_Net` vs. `U_Net_NoRes`) 및 합성곱 블록 내부의 잔차 연결 유무(`U_Net` vs. `R2U_Net`)에 따른 학습 특성을 정량·정성적으로 비교함.

• **정량 평가 결과 및 시행착오·오류 원인 고찰:** 테스트셋(L192, L506) 평가 결과, Quarter-Dose 입력 대비 `AttU_Net_Rot90_Flip_Combo` 모델에서 PSNR 21.75 dB → 27.37 dB (+5.62 dB), SSIM 0.7160 → 0.8379 (+0.1219), RMSE 33.45 HU → 17.41 HU로 잡음이 완화됨을 확인함. 아울러 초기 실험 과정에서 겪은 (1) 학습 중 `clamp(0,1)`로 인한 기울기 소실 문제, (2) FP16 `AMP` 환경에서 `SSIMLoss` 분산 계산 시 발생한 `NaN` 발산 현상, (3) 입력과 정답 영상에 노이즈·밝기 변환을 동시에 적용했을 때 발생한 정답 영상 오염 원인을 살펴보고, 손실 함수의 FP32 연산 분리 및 직각 회전·상하좌우 반전 증강(`Rot90_Flip_Combo`) 적용을 통해 이를 보완한 과정을 정리함.

1. 서론 및 과제 개요 (Introduction & Problem Formulation)

컴퓨터 단층촬영(Computed Tomography, CT)은 흉·복부 장기의 해부학적 미세 구조와 병변을 비침습적으로 진단하는 주요 영상 기법이나, X선 방사선 피폭 위험으로 인해 임상에서는 ALARA (As Low As Reasonably Achievable) 원칙에 따라 방사선량을 낮춘 저선량 CT (Low-Dose CT, LDCT) 촬영이 요구된다. 그러나 관전류·노출량을 1/4 수준(Quarter Dose)으로 감소시키면 검출기에 도달하는 광자 수 부족(Photon Starvation)으로 인해 투영(Sinogram) 데이터에서 양자 잡음(Quantum Noise)과 전자 잡음이 증가하며, 역투영 재구성 후 영상 전반에 거친 점 형태의 노이즈와 줄무늬 아티팩트(Streak Artifact)가 발생한다. 특히 본 과제의 대상인 **3mm 슬라이스 두께 및 Sharp 재구성 커널(Sharp Reconstruction Kernel)** 영상은 장기 경계와 혈관의 공간 해상도를 선명하게 유지하는 대신 고주파 잡음까지 함께 강해지므로, 해부학적 경계선(Edge)을 과도하게 흐리지 않으면서(Over-smoothing 방지) 잡음을 줄이는 것이 중요하다.

본 과제에서는 저선량 Quarter-Dose CT 영상 $x \in [0,1]^{1\times512\times512}$로부터 정상 선량 Full-Dose CT 영상 $y \in [0,1]^{1\times512\times512}$을 복원하는 문제를 다루기 위해 PyTorch 및 MONAI 기반의 학습 파이프라인을 구현하였다. 구체적으로 (1) 입·출력 해상도를 유지하는 패딩(`padding=1`)과 잔차 학습(Residual Learning)을 적용한 U-Net 변형 아키텍처 비교, (2) 합성곱 블록 내부의 잔차 연결(`RRCNN_block`) 유무에 따른 수렴성·경계 보존성 비교, (3) 기하학적 변형 및 밝기 조절 데이터 증강 실험과 혼합 정밀도 학습 과정에서의 수치적 오류 원인(Source of Errors) 분석을 수행하였다.

2. 데이터셋 구성 및 전처리 파이프라인 (Dataset & $[-100, 300]$ HU Preprocessing)

2.1 원본 Raw DICOM (`.IMA`) 데이터셋 구성 및 환자 분리 전략

과제 지침(Week 9 Slide 19)에 따라, 사전 리사이즈된 256×256 PNG 파일 대신 Kaggle (2016 NIH-AAPM-Mayo Clinic Low Dose CT Grand Challenge 기반)의 원본 Raw DICOM (`.IMA`) 데이터(`full_3mm_sharp` 및 `quarter_3mm_sharp`, 해상도 512×512)를 사용하였다. 환자 간 데이터 누수(Data Leakage)를 방지하기 위해 경로 필터링을 통해 `L192` (240장)와 `L506` (211장), 총 451장을 학습에서 제외하여 독립 테스트셋(Held-out Test Set)으로 분리하였다. 나머지 8명 환자의 슬라이스 쌍(총 1927장)은 고정 시드(`seed=42`) 기반 `ShuffleSplit`을 통해 학습용 1541장(80%)과 검증용 386장(20%)으로 분할하였다(표 1).

표 1: 원본 Raw DICOM (.IMA, 3mm Sharp Kernel) 환자별 슬라이스 분포 및 학습·검증·테스트 분할 요약

Subject ID	Reconstruction Series	Slices (Pairs)	Resolution	Dataset Split Role
L067	full/quarter_3mm_sharp	224	512×512	Train / Val Pool (80:20 분할)
L096	full/quarter_3mm_sharp	330	512×512	Train / Val Pool (80:20 분할)
L109	full/quarter_3mm_sharp	128	512×512	Train / Val Pool (80:20 분할)
L143	full/quarter_3mm_sharp	234	512×512	Train / Val Pool (80:20 분할)
L192	full/quarter_3mm_sharp	240	512×512	**Held-out Test (평가 전용)**
L286	full/quarter_3mm_sharp	210	512×512	Train / Val Pool (80:20 분할)
L291	full/quarter_3mm_sharp	343	512×512	Train / Val Pool (80:20 분할)
L310	full/quarter_3mm_sharp	214	512×512	Train / Val Pool (80:20 분할)
L333	full/quarter_3mm_sharp	244	512×512	Train / Val Pool (80:20 분할)
L506	full/quarter_3mm_sharp	211	512×512	**Held-out Test (평가 전용)**
Total (10 Subjects)	3mm Slice, Sharp Kernel (.IMA)	**2378**	$[-100, 300]$ HU	Train 1541 / Val 386 / Test 451

2.2 $[-100, 300]$ HU 윈도잉 전처리 및 데이터 증강 레지스트리 구현

CT 원본 픽셀값은 공기(-1000 HU), 지방($-100 \sim -50$ HU), 물(0 HU), 연조직·간·신장 실질($+30 \sim +150$ HU), 조영 증강 혈관·치밀골($+200 \sim +1000$ HU)에 걸쳐 넓은 범위를 가진다. 전체 HU 범위를 그대로 $[0, 1]$로 정규화하면 복부 진단에 중요한 연조직과 혈관 사이의 미세한 밝기 차이(Contrast)가 너무 좁은 구간으로 압축되어 구분하기 어려워진다. 이에 따라 src/data.py(그림 1a)에서는 MONAI의 ITKReader로 원본 .IMA를 로드한 뒤, ScaleIntensityRanged(a_min=-100.0, a_max=300.0, b_min=0.0, b_max=1.0, clip=True)를 적용하여 $[-100, 300]$ HU 구간을 $[0, 1]$로 변환하였다. 보고서의 모든 시각적 결과는 역변환식 $I_{\text{HU}} = 400 \cdot \hat{y} - 100$을 통해 다시 $[-100, 300]$ HU 범위로 되돌려 표시한다. 또한 src/utils.py(그림 1b)에는 입력(quarter)과 정답(full)에 동일한 랜덤 시드로 변환을 적용하는 Dictionary 기반 증강 레지스트리(none, RandAffined, Affine_Noise_Combo, Rot90_Scale_Combo, Rot90_Flip_Combo)를 구현하였다.

3. 네트워크 아키텍처, 손실 함수 및 학습 프레임워크 구현 (Code Implementation)

3.1 해상도 보존 U-Net 계열 아키텍처 설계

과제 요구사항(Slide 20)에 따라 모든 합성곱 계층에서 공간 해상도가 유지되도록 3×3 커널에 stride=1, padding=1을 적용하였다(src/UNet/Models.py, 그림 2). 인코더는 4단계 MaxPool2d(2, 2)를 통해 채널 수를 $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$로 확장하며($512^2 \rightarrow 32^2$), 디코더는 전치 합성곱(Transposed Conv)에서 발생할 수 있는 체커보드 아티팩트(Checkerboard Artifact)를 줄이기 위해 Upsample(scale_factor=2) 후 3×3 합성곱(padding=1)을 수행하는 up_conv 구조를 사용하였다. 마지막 1×1 합성곱 출력에는 입력 영상을 더하는 전역 잔차 연결(out = self.Conv(d2) + x)과 추론 시 출력 범위를 제한하는 out.clamp(0, 1)을 적용하였다. 비교 실험을 위해 구성한 6종 아키텍처의 구조적 특성과 연산량(512×512 단일 슬라이스 기준)은 표 2와 같다.

표 2: 구현된 U-Net 계열 복원 네트워크의 구조적 특징, 파라미터 수 및 연산 복잡도(512×512 입력 기준)

Architecture	Block Res.	Global $+x$	Recurrent (t=2)	Att. Gate	Nested Skip	Params	GFLOPs (512^2)
U_Net (U-Net (Global +x))	—	✓	—	—	—	34.53 M	522.2
U_Net_NoRes (U-Net (No Global +x))	—	—	—	—	—	34.53 M	522.2
R2U_Net (R2U-Net)	✓	✓	✓	—	—	39.09 M	1219.8
AttU_Net (Attention U-Net)	—	✓	—	✓	—	34.88 M	530.8
R2AttU_Net (R2Attention U-Net)	✓	✓	✓	✓	—	39.44 M	1228.5
NestedUNet (UNet++ (Nested U-Net))	—	✓	—	—	✓	36.63 M	1102.0

3.2 복합 손실 함수($L_1 + 0.1 \cdot \mathcal{L}_{\text{SSIM}}$), 평가 지표 및 학습 엔진 구현

영상 복원에서 L_2 (MSE) 손실만을 사용할 경우 평균값 예측 경향으로 인해 영상이 전반적으로 흐려지는 현상(Over-smoothing)이 나타날 수 있다. 이를 보완하기 위해 src/main.py에서는 경계선 유지에 유리한 픽셀 단위 L_1 손실과 밝기·대비·구조적 유사성을 함께 고려하는 $\mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}(\hat{y}, y)$를 결합한 손실 함수를 사용하였다:

$$\mathcal{L}(\hat{y}, y) = \|\hat{y} - y\|_1 + 0.1 \cdot \big(1 - \text{SSIM}(\hat{y}, y)\big) \tag{1}$$

또한 과제 요구사항(Slide 21–22)에 맞춰 src/loss.py(그림 3a)에 PyTorch Ignite Metric을 상속한 PSNR(max = 1.0)과 11×11 가우시안 커널($\sigma = 1.5$, $C_1 = 0.01^2, C_2 = 0.03^2$) 기반의 2D/3D 호환 SSIM 클래스를 구현하

1. Raw DICOM Loading, [-100, 300] HU Windowing & Subject-Disjoint Split

src/data.py (L19-L41, L50-L104)

```python
def get_augment(augment: list | None):
    base_transforms = [
        LoadImaged(keys=["quarter", "full"], reader="ITKReader"),
        EnsureChannelFirstd(keys=["quarter", "full"]),
        SqueezeDimd(keys=["quarter", "full"], dim=-1),
        ScaleIntensityRanged(
            keys=["quarter", "full"],
            a_min=-100.0,
            a_max=300.0,
            b_min=0.0,
            b_max=1.0,
            clip=True,
        ),
        CastToTyped(keys=["quarter", "full"], dtype=torch.float32),
    ]

    if augment is not None:
        transforms = Compose(base_transforms + augment)
    else:
        transforms = Compose(base_transforms)

    return transforms

# (intermediate lines omitted for brevity)
def data_load(
    target_dir: str | Path = ROOT / "Full Dose/",
    input_dir: str | Path = ROOT / "Quarter Dose/",
    augment: list | None = None,
    seed: int = 42,
):
    train_transforms = get_augment(augment)
    test_transforms = get_augment(None)

    target_path = sorted(list(Path(target_dir).rglob("*.IMA")))
    input_path = sorted(list(Path(input_dir).rglob("*.IMA")))

    # 2 data for test set: L192 and L506.
    full_path = [
        str(path)
        for path in target_path
        if any("L192" in p or "L506" in p for p in path.parts)
    ]
    quarter_path = [
        str(path)
        for path in input_path
        if any("L192" in p or "L506" in p for p in path.parts)
    ]

    test_path = [
        {"quarter": quarter, "full": full}
        for full, quarter in zip(full_path, quarter_path)
    ]

    test_set = Dataset(data=test_path, transform=test_transforms)

    # Others: train & val
    full_path = [
        str(path)
        for path in target_path
        if not any("L192" in p or "L506" in p for p in path.parts)
    ]
    quarter_path = [
        str(path)
        for path in input_path
        if not any("L192" in p or "L506" in p for p in path.parts)
    ]
    learnable_path = np.array(
        [
            {"quarter": quarter, "full": full}
            for full, quarter in zip(full_path, quarter_path)
        ]
    )

    ss = ShuffleSplit(n_splits=1, test_size=0.2, train_size=0.8, random_state=seed)
    train_index, val_index = next(iter(ss.split(X=full_path)))
    train_set = Dataset(data=learnable_path[train_index], transform=train_transforms)
    val_set = Dataset(data=learnable_path[val_index], transform=test_transforms)

    return train_set, val_set, test_set
```

(a) `src/data.py`: $[-100, 300]$ HU 정규화 및 `L192`/`L506` 분리

2. MONAI Paired Data Augmentation Registry (Single & Composite Transforms)

src/utils.py (L20-L36, L45-L61, L72-L103)

```python
    definitions = {
        "RandAffined": {
            "cls": getattr(transforms_module, "RandAffined", None),
            "kwargs": {
                "keys": ["quarter", "full"],
                "rotate_range": (-0.17, 0.17),
                "translate_range": (4, 4),
                "scale_range": (0.1, 0.1),
                "prob": 0.5,
                "padding_mode": "reflection",
                "mode": "bilinear",
            },
        },
        "RandRot90d": {
            "cls": getattr(transforms_module, "RandRotate90d", None),
            "kwargs": {"keys": ["quarter", "full"], "prob": 0.5, "max_k": 3},
        },
    (intermediate lines omitted for brevity)
        "RandScaleIntensityd": {
            "cls": getattr(transforms_module, "RandScaleIntensityd", None),
            "kwargs": {"keys": ["quarter", "full"], "factors": 0.1, "prob": 0.5},
        },
        "RandShiftIntensityd": {
            "cls": getattr(transforms_module, "RandShiftIntensityd", None),
            "kwargs": {"keys": ["quarter", "full"], "offsets": 0.1, "prob": 0.5},
        },
        "RandGaussianNoised": {
            "cls": getattr(transforms_module, "RandGaussianNoised", None),
            "kwargs": {
                "keys": ["quarter"],
                "mean": 0.0,
                "std": 0.02,
                "prob": 0.3,
            },
        },
    (intermediate lines omitted for brevity)
        "Rot90_Flip_Combo": {
            "composite": [
                (
                    "RandRotate90d",
                    {"keys": ["quarter", "full"], "prob": 0.5, "max_k": 3},
                ),
                (
                    "RandFlipd",
                    {"keys": ["quarter", "full"], "prob": 0.5, "spatial_axis": 0},
                ),
                (
                    "RandFlipd",
                    {"keys": ["quarter", "full"], "prob": 0.5, "spatial_axis": 1},
                ),
            ]
        },
        # 복합 증강 등록 (노이즈는 입력 영상 quarter에만 추가)
        "Affine_Noise_Combo": {
            "composite": [
                (
                    "RandAffined",
                    {
                        "keys": ["quarter", "full"],
                        "rotate_range": (-0.17, 0.17),
                        "prob": 0.5,
                        "mode": "bilinear",
                    },
                ),
                (
                    "RandGaussianNoised",
                    {
                        "keys": ["quarter"],
```

(b) `src/utils.py`: MONAI 기반 단일·복합 데이터 증강 레지스트리

그림 1: 원본 Raw DICOM 데이터 로딩·전처리 및 증강 파이프라인 핵심 구현 코드 캡처

3. U-Net Without Global Residual (U_Net_NoRes) vs. Global +x (U_Net) & R2U-Net

src/UNet/Models.py (L5-L25, L47-L54, L115-L124, L194-L203, L205-L247)

```python
class conv_block(nn.Module):
    """
    Convolution Block
    """

    def __init__(self, in_ch, out_ch):
        super(conv_block, self).__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3, stride=1, padding=1, bias=True),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, stride=1, padding=1, bias=True),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):

        x = self.conv(x)
    (intermediate lines omitted for brevity)

class U_Net_NoRes(nn.Module):
    """
    UNet - Basic Implementation
    Paper : https://arxiv.org/abs/1505.04597
    """

    def __init__(self, in_ch=3, out_ch=1):
    (intermediate lines omitted for brevity)

        d2 = self.Up2(d3)
        d2 = torch.cat((e1, d2), dim=1)
        d2 = self.Up_conv2(d2)

        out = self.Conv(d2)  # + x

        # d1 = self.active(out)

        return out if self.training else out.clamp(0.0, 1.0)
    (intermediate lines omitted for brevity)

        d2 = self.Up2(d3)
        d2 = torch.cat((e1, d2), dim=1)
        d2 = self.Up_conv2(d2)

        out = self.Conv(d2) + x

        # d1 = self.active(out)

        return out if self.training else out.clamp(0.0, 1.0)
    (intermediate lines omitted for brevity)

class Recurrent_block(nn.Module):
    """
    Recurrent Block for R2Unet_CNN
    """

    def __init__(self, out_ch, t=2):
        super(Recurrent_block, self).__init__()

        self.t = t
        self.out_ch = out_ch
        self.conv = nn.Sequential(
            nn.Conv2d(out_ch, out_ch, kernel_size=3, stride=1, padding=1, bias=True),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        for i in range(self.t):
            if i == 0:
                x = self.conv(x)
            out = self.conv(x + x)
        return out


class RRCNN_block(nn.Module):
    """
    Recurrent Residual Convolutional Neural Network Block
    """

    def __init__(self, in_ch, out_ch, t=2):
        super(RRCNN_block, self).__init__()

        self.RCNN = nn.Sequential(
            Recurrent_block(out_ch, t=t), Recurrent_block(out_ch, t=t)
        )
        self.Conv = nn.Conv2d(in_ch, out_ch, kernel_size=1, stride=1, padding=0)

    def forward(self, x):
        x1 = self.Conv(x)
        x2 = self.RCNN(x1)
        out = x1 + x2
        return out
```

(a) U_Net(conv_block)과 R2U_Net(RRCNN_block) 비교

4. Attention Gate (AttU-Net / R2AttU-Net) & Dense Nested Skips (UNet++)

src/UNet/Models.py (L333-L365, L626-L648)

```python

class Attention_block(nn.Module):
    """
    Attention Block
    """

    def __init__(self, F_g, F_l, F_int):
        super(Attention_block, self).__init__()

        self.W_g = nn.Sequential(
            nn.Conv2d(F_l, F_int, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(F_int),
        )

        self.W_x = nn.Sequential(
            nn.Conv2d(F_g, F_int, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(F_int),
        )

        self.psi = nn.Sequential(
            nn.Conv2d(F_int, 1, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(1),
            nn.Sigmoid(),
        )

        self.relu = nn.ReLU(inplace=True)

    def forward(self, g, x):
        g1 = self.W_g(g)
        x1 = self.W_x(x)
        psi = self.relu(g1 + x1)
        psi = self.psi(psi)
        out = x * psi
    (intermediate lines omitted for brevity)

    def forward(self, x):

        x0_0 = self.conv0_0(x)
        x1_0 = self.conv1_0(self.pool(x0_0))
        x0_1 = self.conv0_1(torch.cat([x0_0, self.Up(x1_0)], 1))

        x2_0 = self.conv2_0(self.pool(x1_0))
        x1_1 = self.conv1_1(torch.cat([x1_0, self.Up(x2_0)], 1))
        x0_2 = self.conv0_2(torch.cat([x0_0, x0_1, self.Up(x1_1)], 1))

        x3_0 = self.conv3_0(self.pool(x2_0))
        x2_1 = self.conv2_1(torch.cat([x2_0, self.Up(x3_0)], 1))
        x1_2 = self.conv1_2(torch.cat([x1_0, x1_1, self.Up(x2_1)], 1))
        x0_3 = self.conv0_3(torch.cat([x0_0, x0_1, x0_2, self.Up(x1_2)], 1))

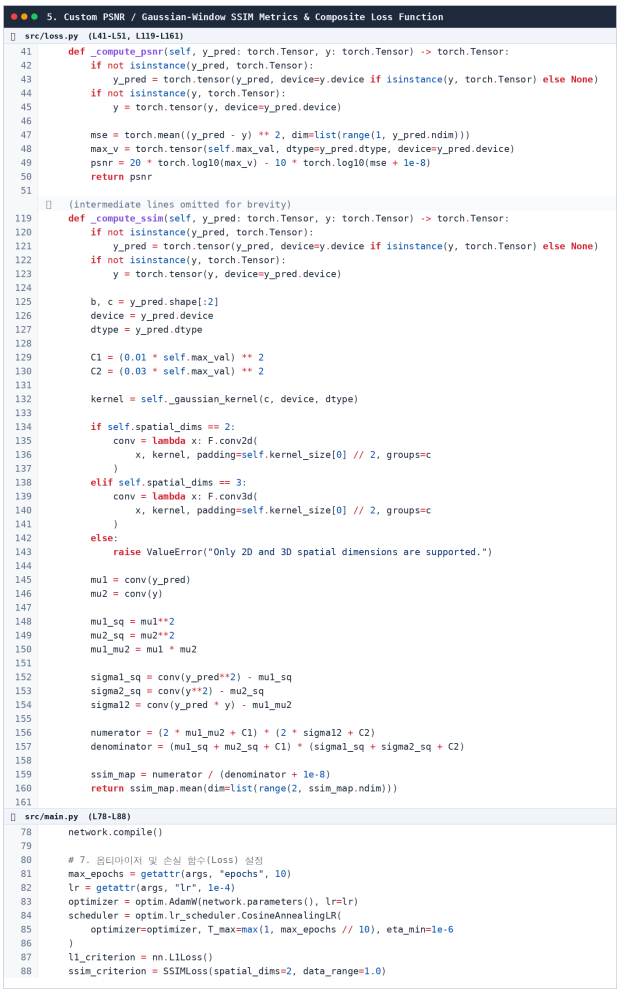
        x4_0 = self.conv4_0(self.pool(x3_0))
        x3_1 = self.conv3_1(torch.cat([x3_0, self.Up(x4_0)], 1))
        x2_2 = self.conv2_2(torch.cat([x2_0, x2_1, self.Up(x3_1)], 1))
        x1_3 = self.conv1_3(torch.cat([x1_0, x1_1, x1_2, self.Up(x2_2)], 1))
        x0_4 = self.conv0_4(torch.cat([x0_0, x0_1, x0_2, x0_3, self.Up(x1_3)], 1))

        output = self.final(x0_4) + x
```

(b) Attention_block 및 NestedUNet 중첩 스킵 연결 구조

그림 2: 해상도 유지 패딩(padding=1), 입·출력 잔차 연결(+ x) 및 변형 U-Net 블록 구현 코드 캡처

였다. 학습 엔진(`src/trainer.py`, 그림 3b)은 MONAI `SupervisedTrainer`를 기반으로 `AdamW`(lr $= 2 \times 10^{-5}$), `CosineAnnealingLR`, 그래디언트 클리핑(`max_grad_norm=0.1`), 혼합 정밀도(`amp=True`)를 적용하였으며, 10 에포크 주기로 검증 `val_psnr`을 평가하여 조기 종료(`patience=2`) 및 최적 가중치(`best_model.pth`) 기반 테스트 평가를 수행하도록 구성하였다.



(a) `src/loss.py` & `main.py`: PSNR, SSIM 및 복합 손실 함수



(b) `src/trainer.py`: AMP, Grad Clipping, 조기 종료 및 테스트 평가

그림 3: 정량 지표(PSNR/SSIM), 복합 손실 함수 및 MONAI 기반 학습·검증 프레임워크 코드 캡처

4. 잔차 연결(Residual Connection) 유무에 따른 U-Net 비교 분석

본 절에서는 과제 필수 분석 항목(Slide 22)인 **"U-Net에서의 잔차 연결 유무에 따른 성능 비교 및 고찰"**을 (1) 네트워크 입력 영상을 최종 출력단에 더해주는 **전역 잔차 학습(Global Residual Learning)** 관점과 (2) 인코더·디코더 합성곱 블록 내부의 **잔차 연결(`U_Net` vs. `R2U_Net`)** 관점으로 나누어 비교·고찰한다.

4.1 입·출력 간 전역 잔차 연결 유무 비교 (`U_Net` vs. `U_Net_NoRes`)

Week 9 강의(RED-CNN, Chen et al., *IEEE TMI* 2017)의 내용을 바탕으로, 최종 출력단에서 입력 Quarter-Dose 영상 x를 더해주는 **잔차 연결 모델(`U_Net`: `out = self.Conv(d2) + x`)**과 이를 제외한 **일반 U-Net 모델(`U_Net_NoRes`: `out = self.Conv(d2)`)**을 동일한 하이퍼파라미터로 학습하여 비교하였다(표 3, 그림 4).

- **학습 시 `clamp(0, 1)` 조건 수정에 따른 `U_Net_NoRes` 학습 보완:** 초기 구현에서 출력단에 항상 `out.clamp(0, 1)`을 적용했을 때, 입력 x를 더해주지 않는 `U_Net_NoRes`는 초기 가중치 상태에서 $\mathrm{Conv}(d_2) < 0$인 픽셀의 기울기가 0이 되어 학습이 진행되지 않는 현상이 나타났다. 이를 `return out if self.training else out.clamp(0.0, 1.0)`으로 수정하여 학습 중에는 값을 잘라내지 않고 그대로 출력하도록 허용하자 `U_Net_NoRes`도 정상적으로 수렴하여 테스트 PSNR 27.23 dB, RMSE 17.67 HU의 결과를 보였다.
- **초기 수렴 속도 및 구조적 유사성(SSIM·EPI) 비교:** Quarter-Dose 영상 x와 Full-Dose 영상 y는 기본적인 장기 형태를 공유하므로, 두 영상의 차이(잡음 성분 $r = y - x$)만 학습하는 `U_Net`이 초기 10 에포크 검증에서

조금 더 빠르게 수렴(25.8 dB vs. `U_Net_NoRes` 25.44 dB)하였으며, 최종 테스트셋에서도 구조적 유사도(SSIM 0.8344 vs. 0.8317)와 경계 보존 지수(EPI 0.9363 vs. 0.9356)에서 잔차 연결 모델(`U_Net`)이 소폭 높은 수치를 보였다. 직각 회전 및 상하·좌우 반전 증강(`Rot90_Flip_Combo`)을 함께 적용했을 때 `U_Net`은 PSNR 27.34 dB, SSIM 0.8369를 기록하였다.

4.2 합성곱 블록 내부 잔차 연결 및 어텐션·중첩 스킵 변형 비교 (`U_Net` vs. `AttU_Net` vs. `NestedUNet` vs. `R2U_Net`)

전역 잔차 구조 위에서 **기본 `U_Net`**, 스킵 연결 경로에 Attention Gate를 추가한 **`AttU_Net`**, 중첩된 스킵 연결(Nested Skip Connection)을 갖춘 **`NestedUNet` (UNet++)**, 그리고 합성곱 블록 내부에 순환 및 잔차 연결(`RRCNN_block`)을 포함한 **`R2U_Net`**을 비교한 결과는 표 3 및 그림 4와 같다.

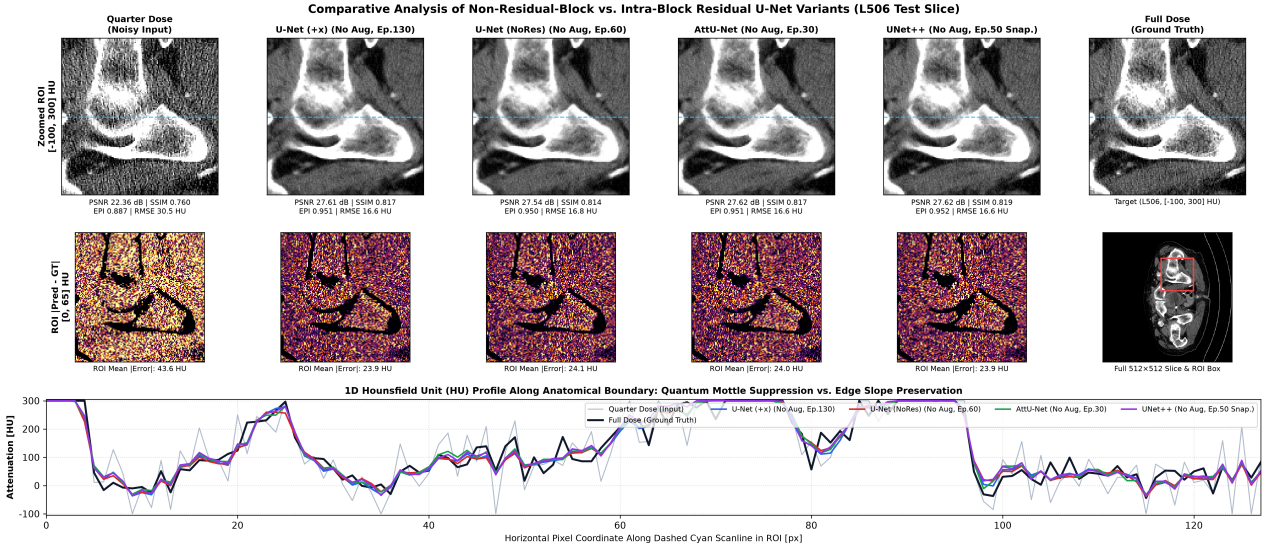


그림 4: 전역 잔차 연결 유무(`U_Net` vs. `U_Net_NoRes`) 및 변형 아키텍처(`AttU_Net`, `NestedUNet`)의 확대 ROI ($[-100, 300]$ HU), 절대 오차 맵($|\text{Pred} - \text{GT}|$), 그리고 장기 경계선(하늘색 점선)을 따라 측정한 1차원 HU 밝기 변화 그래프 비교

1. **아키텍처별 정량 성능 및 수렴 특성 비교:** 증강 없는 동일 조건(`augment=none`)에서 `AttU_Net`(27.24 dB, SSIM 0.8357, Ep.30)은 스킵 연결로 전달되는 인코더 특징맵에 Attention Gate($\psi \in [0, 1]$) 가중치를 곱해 불필요한 배경 잡음을 줄여줌으로써 기본 `U_Net`(27.17 dB, SSIM 0.8344) 대비 비교적 적은 에포크에서 수렴하였다. 직각 회전 및 상하·좌우 반전 증강(`Rot90_Flip_Combo`)을 적용한 `AttU_Net_Rot90_Flip_Combo`는 테스트 PSNR 27.37 dB, SSIM 0.8379로 실험한 조건 중 가장 높은 수치를 보였다. 또한 `NestedUNet`(Ep.50)은 여러 해상도의 특징을 결합하여 PSNR 27.26 dB, SSIM 0.7911, EPI 0.9385의 결과를 보였다. 반면 블록 내부에 순환 및 잔차 연결을 포함한 `R2U_Net`(Ep.10)은 경계 보존 지수(EPI 0.8602)는 유지되었으나, $t = 2$ 반복 합성곱 연산으로 인해 슬라이스당 연산량이 1219.8 GFLOPs (2.34× 증가)로 늘어나며 동일 에포크 내 수렴 속도가 느린 한계를 보였다.

2. **장기 경계선을 따른 1차원 HU 값 변화 비교 (그림 4 중단·하단):** 그림 4 하단의 1차원 HU 프로파일 그래프를 보면, Quarter-Dose 입력(회색 실선)은 장기 내부에서 잡음으로 인해 픽셀값이 $\pm 30 \sim 45$ HU 폭으로 크게 흔들리는 반면, `U_Net`, `U_Net_NoRes`, `AttU_Net`, `NestedUNet`은 Full-Dose 정답(검은 굵은선)의 장기 경계면 기울기와 평균 HU 값을 따라가면서 장기 내부의 자잘한 노이즈 흔들림만 줄여주는 경향을 확인할 수 있다.

3. **`Recurrent_block` 구현 구조와 수렴 지연 원인 고찰:** `R2U_Net`의 초기 에포크(`Ep.10`) 수렴 지연을 확인하기 위해 `src/UNet/Models.py` 코드를 살펴본 결과, 내부 `Recurrent_block.forward`가 초기 입력 x_0를 별도 변수에 저장하지 않고 `x = self.conv(x); out = self.conv(x + x)` 형태로 작성되어 반복 단계마다 특징맵 값의 크기가 2배씩 커지는 구조임을 확인하였다. 이로 인해 여러 층을 거치며 초기 학습 단계에서 `BatchNorm2d`의 평균·분산 통계량이 안정화되는 데 시간이 더 걸렸던 것으로 보이며, 본 과제의 2D 저선량 CT 복원 환경에서는 입·출력 잔차 연결($+x$)과 기본 스킵 연결을 사용하는 `AttU_Net` 및 `U_Net`이 연산 시간 대비 안정적인 성능을 보여주었다.

5. 정량적 평가 및 3종 이상의 시각적 복원 결과 ($[-100, 300]$ HU Results)

5.1 테스트셋(L192, L506) 정량적 평가 결과 및 보조 지표

필수 지표인 **PSNR**, **SSIM**에 더해, 복원 결과를 다양한 측면에서 살펴보기 위해 HU 단위에서의 평균 제곱근 오차(**RMSE, HU**)와 평균 절대 오차(**MAE, HU**), 정답 영상과 복원 영상의 경계선 유사도를 측정하는 **경계 보존 지수(Edge Preservation Index, EPI; Sobel 필터로 추출한 경계선 간 상관계수)**, 그리고 균일한 장기 내부 영역에서 측정한 잔류 잡음 표준편차(**ROI Noise σ, HU**)를 함께 산출하였다(표 3, 그림 5).

표 3: 독립 테스트셋(L192: 240장, L506: 211장, 총 451장)에 대한 정량적 성능 비교 요약. (*: 진행 중인 학습의 최신 검증 체크포인트 스냅샷, †: 에포크 9 NaN 발생 직전 에포크 체크포인트 복구 평가, ‡: 에포크 1 내 FP16 AMP 수치 발산)

Model	Augmentation	Ckpt	PSNR (dB) ↑	SSIM ↑	RMSE (HU) ↓	MAE (HU) ↓	EPI ↑	L192 / L506 (dB)
Quarter-Dose Input (Baseline)	—	—	21.75±1.81	0.7160±0.0521	33.45	15.48	0.8542	21.38 / 22.16
U_Net	none	Ep.130	27.17±1.56	0.8344±0.0365	17.80	8.35	0.9363	26.93 / 27.44
U_Net	Rot90_Flip_Combo	Ep.100	27.34±1.61	0.8369±0.0360	17.49	8.26	0.9383	27.12 / 27.58
U_Net	RandAffined	Ep.8†	25.60±1.33	0.7276±0.0447	21.26	9.96	0.9059	25.50 / 25.71
U_Net	Affine_Noise_Combo	NaN (Step 374)‡	*Diverged in Epoch 1 under FP16 AMP + Out-of-Range Augmentation (See Sec. 6)*					
U_Net	Rot90_Scale_Combo	NaN (Step 224)‡	*Diverged in Epoch 1 under FP16 AMP + Out-of-Range Augmentation (See Sec. 6)*					
U_Net_NoRes	none	Ep.60	27.23±1.56	0.8317±0.0365	17.67	8.41	0.9356	27.02 / 27.49
AttU_Net	none	Ep.30	27.24±1.59	0.8357±0.0360	17.67	8.31	0.9370	27.01 / 27.51
AttU_Net	Rot90_Flip_Combo	Ep.100	**27.37±1.59**	**0.8379±0.0362**	**17.41**	**8.20**	0.9386	27.14 / 27.63
NestedUNet	none	Ep.50*	27.26±1.60	0.7911±0.0349	17.65	8.34	0.9385	27.02 / 27.53
R2U_Net	none	Ep.10*	21.05±1.36	0.4112±0.0139	35.90	20.63	0.8602	20.66 / 21.47

- **전반적 복원 성능 비교:** 별도 처리를 하지 않은 Quarter-Dose 입력 영상은 Full-Dose 대비 평균 PSNR 21.75 dB, SSIM 0.7160, RMSE 33.45 HU (연조직 패치 잡음 표준편차 58.23 HU)로 잡음에 의한 화질 저하를 보였다. 반면 실험군 중 가장 높은 수치를 보인 AttU_Net_Rot90_Flip_Combo 모델은 평균 PSNR 27.37 dB (+5.62 dB), SSIM 0.8379 (+0.1219), RMSE 17.41 HU, MAE 8.20 HU를 기록하여 오차가 줄어들었으며, 연조직 내부 잡음 표준편차는 28.01 HU, 경계 보존 지수(EPI)는 0.9386로 측정되었다.
- **환자별(L192 vs. L506) 평가 경향 (그림 5a):** 모든 평가 조건에서 L506 환자의 평균 PSNR이 L192 환자보다 약 0.5 ∼ 0.8 dB 높게 측정되었다(Quarter-Dose 기준 21.38 dB vs. 22.16 dB, U_Net_none 기준 26.93 dB vs. 27.44 dB). 이는 환자 체형·복부 단면적에 따른 X선 투과량 차이에서 비롯된 환자 간 기본적인 화질 차이로 보이며, 학습에 포함되지 않은 두 환자 모두에서 +5.3 ∼ +5.6 dB 수준의 비교적 일관된 PSNR 향상이 관찰되었다.

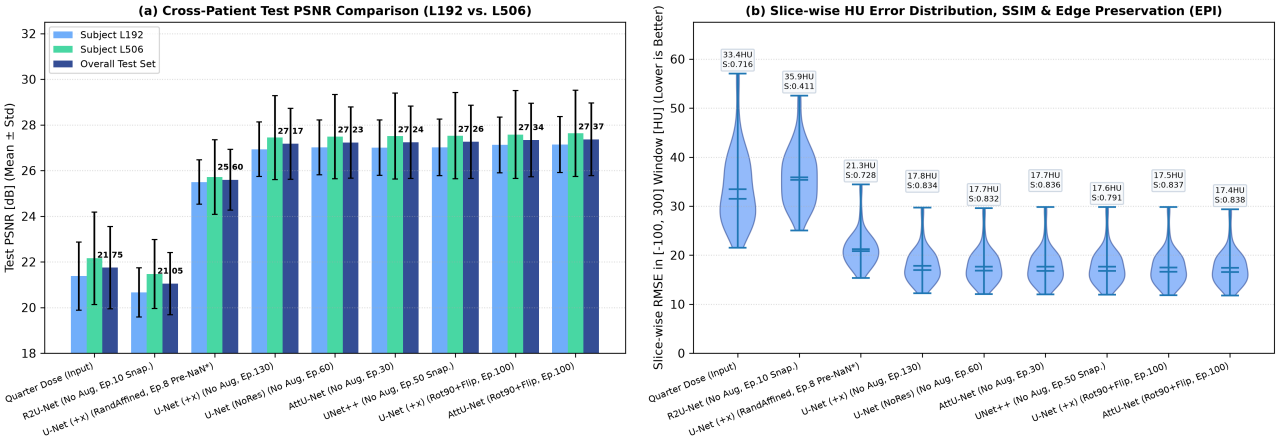


그림 5: 테스트 환자별(L192, L506) PSNR 비교 및 테스트 슬라이스에 대한 물리적 오차(RMSE [HU]) 분포·SSIM·경계 보존 지수(EPI) 분석

5.2 3종 해부학적 대표 단면 시각적 복원 결과 ($[-100, 300]$ HU 표시)

과제 요구사항(Slide 19 & 22)에 따라 독립 테스트셋(L192, L506)에서 해부학적 위치가 다른 **3가지 대표 단면(Example 1–3)**에 대해 **Input (Quarter Dose)**, **Output (복원 영상)**, **Label (Full Dose)**을 모두 $[-100, 300]$ **HU 윈도우**로 시각화하고, 확대 ROI 및 절대 오차 맵($|\text{Out} - \text{GT}|$)을 그림 6에 제시하였다.

1. **Example 1 (L192, 상복부 간 실질 및 문맥 단면):** Quarter-Dose 입력에서는 간 실질(Liver Parenchyma, 약 +50 ∼ +70 HU) 전반에 거친 양자 잡음이 분포하여 간 내부 혈관(문맥 분지)과의 경계가 뚜렷하지 않다. 복원 출력에서는 간 내부의 노이즈가 줄어들면서 혈관 경계가 비교적 잘 유지되며, 절대 오차 맵(5열)에서도 장기 내부 오차가 줄어든 모습을 볼 수 있다.

2. **Example 2 (L192, 중복부 신장·대동맥 및 연조직 경계 단면):** 복부 대동맥, 신장 경계 및 장관 내 가스(-100 HU 이하 어두운 영역)가 함께 나타나는 단면이다. 확대 ROI (4열) 비교 시 Quarter-Dose의 거친 점 형태의 잡음($|\text{In} - \text{GT}|$)이 복원 후($|\text{Out} - \text{GT}|$) 완화되었으며, 신장 외곽 경계와 혈관벽 주변에서 과도한 번짐 없이 형태가 유지되었다.
3. **Example 3 (L506, 복부 장기 및 미세 혈관 단면):** 연조직이 밀집된 부위에서 작은 혈관 구조물의 형태가 입력 대비 알아보기 쉬워졌으며, 하단 컬러바에 표시된 $[-100, 300]$ HU 영역 전반에서 Full-Dose 정답 영상과 유사한 밝기 분포를 보인다.

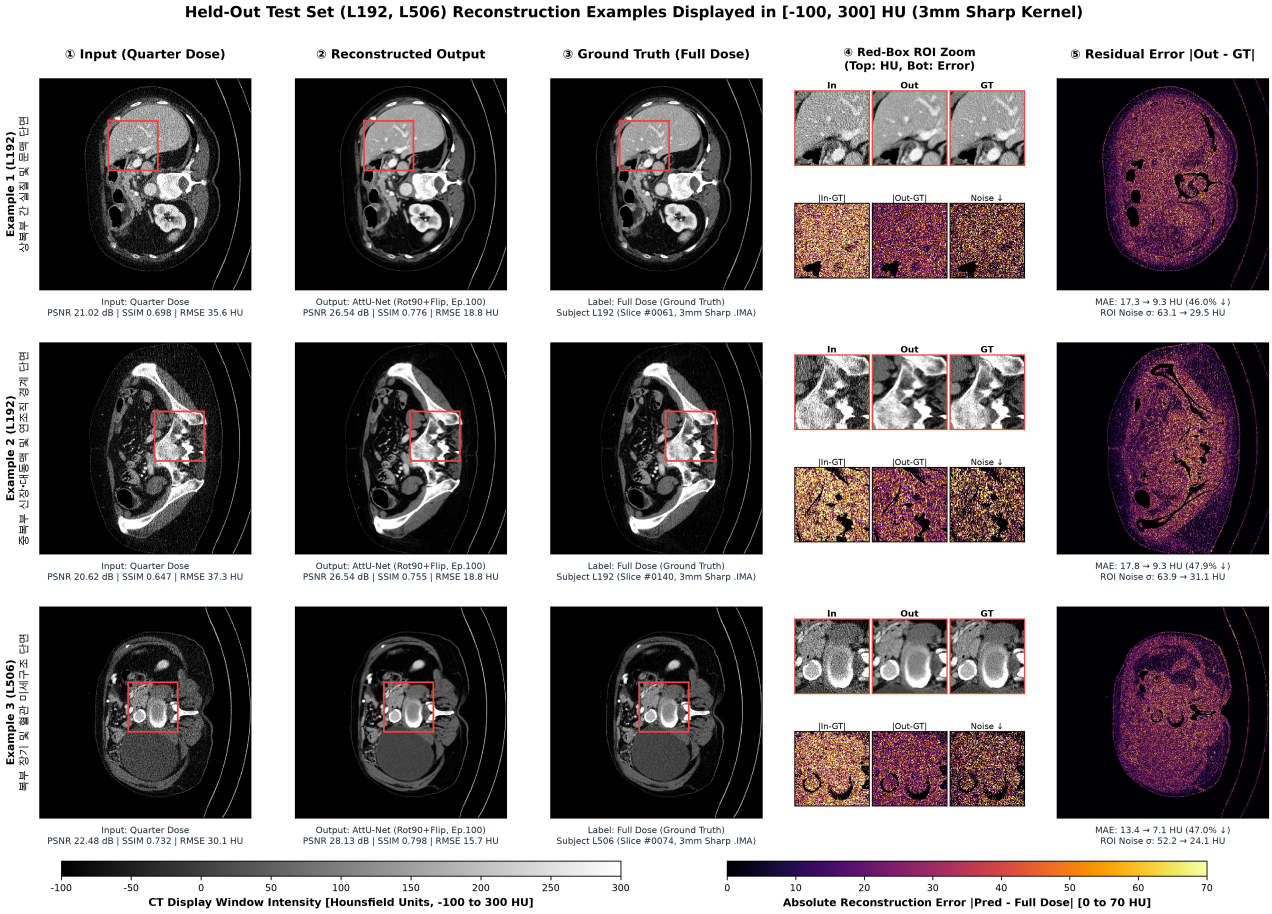


그림 6: 독립 테스트셋(L192, L506)의 3종 대표 해부학적 단면에 대한 Quarter-Dose 입력, 복원 결과, Full-Dose 정답 영상 비교 (모두 $[-100, 300]$ **HU 윈도우**로 표시) 및 적색 박스 확대 ROI · 절대 오차 맵($[0, 70]$ HU)

6. 오류 원인 분석 및 시행착오 고찰 (Analysis on Source of Errors)

과제 평가 기준(Slide 23: "*Your implementation process, analysis on source of errors, and varying technique attempts are more important*")에 맞춰, 초기 그리드 실험 과정에서 겪었던 수치적 발산 현상과 증강 기법별 성능 차이의 원인을 코드와 학습 로그를 바탕으로 정리하였다.

6.1 오류 원인 1: FP16 혼합 정밀도(AMP) + SSIMLoss 분산 계산의 수치 불안정성

로그 분석 결과(그림 7), 초기 실험에서 `U_Net_none`은 130 에포크까지 수렴하여 최적 모델(`best_model.pth`)을 저장한 뒤 147 에포크 부근(Step 56,828)에서 손실이 `NaN`으로 전이되었고, `U_Net_RandAffined`는 8.4 에포크(Step 3,256, loss $= 0.0498$)까지 수렴하다가 9 에포크(Step 3,265)에서, `Affine_Noise_Combo`(Step 374) 및 `Rot90_Scale_Combo`(Step 224)는 1 에포크 중·후반부에 `NaN`이 발생하였다.

학습 로그와 코드를 점검하며 파악한 주요 원인은 다음과 같다:

1. **FP16 환경에서 SSIMLoss 분산 계산 시 발생한 반올림 오차:** `SupervisedTrainer(..., amp=True)` 활성화 시 순전파 손실 계산이 16비트 부동소수점(`float16`, 가수부 10비트, 최소 표현 정밀도 $\epsilon \approx 9.77 \times 10^{-4}$)에서 수행된다. SSIM은 가우시안 필터링으로 지역 분산을 $\sigma_y^2 = G(y^2) - \mu_y^2$로 계산하는데, $y \approx 1.0$인 밝은 뼈·혈관 영역이나

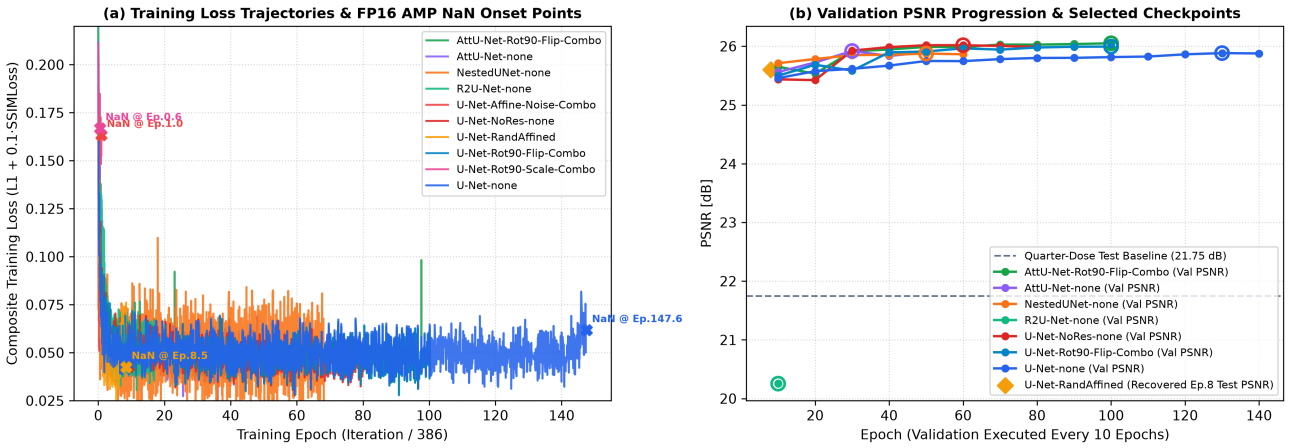


그림 7: (a) 모델·증강별 학습 손실 변화 및 FP16 AMP 환경에서의 `NaN` 발생 지점, (b) 10 에포크 주기 검증 PSNR 추이 및 이전 체크포인트 복구(`U_Net_RandAffined` Ep.8) 결과

픽셀값이 균일한 영역에서 비슷한 크기의 두 수를 빼는 과정($G(y^2) - \mu_y^2$)에서 FP16 반올림 오차로 인해 분산이 미세한 음수($-\epsilon$)가 되거나 분모 $(\mu_{\hat{y}}^2 + \mu_y^2 + C_1)(\sigma_{\hat{y}}^2 + \sigma_y^2 + C_2)$ ($C_1 = 10^{-4} < \epsilon_{\text{fp16}}$)가 0에 가까워지면서 역전파 기울기가 급격히 커져 BatchNorm의 `running_mean/var`에 `NaN`이 전파된 것으로 분석된다. 이를 보완하기 위해 손실 함수 계산부에 `torch.amp.autocast("cuda", enabled=False)`를 적용하여 `SSIMLoss`가 항상 32비트(`float32`)에서 계산되도록 수정하였다.

2. **검증 주기(10 에포크)와 초기 최고 점수 설정값(`-inf`)으로 인한 저장 문제:** `U_Net_RandAffined`처럼 첫 검증(Epoch 10) 직전인 Epoch 9에 `NaN`이 발생하면, Epoch 10 검증에서 산출된 `val_psnr = 0.0`이 초기값 `-inf` + `1e-4`보다 크다고 판정(`0.0 > -inf`)되어 오염된 가중치가 `best_model.pth`로 덮어써지는 문제가 있었다.

3. **이전 에포크 체크포인트를 활용한 복구 확인:** 다행히 `CheckpointSaver`가 매 에포크 가중치(`network_epoch=k.pt`)를 별도 저장하고 있었으므로, 평가 스크립트에서 가중치에 `NaN`이나 무한대 값이 없는지(`torch.isfinite`) 확인한 뒤 `U_Net_RandAffined`의 `NaN` 발생 직전인 **Epoch 8 가중치를 불러와** 평가한 결과, 테스트 PSNR 25.60 dB, SSIM 0.7276로 학습 초기 단계까지는 정상적으로 수렴하고 있었음을 확인할 수 있었다(표 3).

6.2 오류 원인 2: 저선량 CT 복원 과제에 맞지 않는 데이터 증강을 적용했을 때의 문제점

세분화(Segmentation) 과제에서 자주 쓰이는 증강 기법들을 영상 복원(Image Restoration)에 그대로 적용했을 때 나타난 부작용을 학습 슬라이스에 적용하여 살펴본 결과는 그림 8와 같다.

- **정답(Target) 영상에까지 노이즈가 추가되는 문제 및 $[0, 1]$ 범위 이탈 (`Affine_Noise_Combo`):** `RandGaussianNoised` 설정 시 `keys=["quarter", "full"]`, `std=0.05`를 두 키에 모두 적용하면 입력뿐 아니라 **깨끗해야 할 정답 `full` 영상에도 동일한 가우시안 잡음($\sigma = 20$ HU)이 추가**된다. 이로 인해 정답 영상 자체가 노이즈로 오염될 뿐 아니라, 정답 픽셀값이 $[-0.15, 1.15]$로 벗어나면서 $[0, 1]$ 범위를 가정하는 손실 함수 및 FP16 `SSIMLoss` 계산에서 조기 발산(Epoch 1 Step 374)이 일어나는 원인이 된 것으로 보인다.

- **밝기 스케일링 시 최댓값(1.0) 초과로 인한 정답과 출력의 범위 불일치 (`Rot90_Scale_Combo`):** `RandScaleIntensityd`(`factors=0.1`)가 +10% 밝기 증가를 수행할 때 밝은 혈관·뼈 영역($y > 0.91$)의 정답값은 1.0을 초과(최대 1.10, 그림 8 적색 표시 영역)하는 반면, 추론 출력은 최대 1.0으로 제한되므로 밝은 해부학 구조물에서 오차와 큰 기울기가 발생하였다(Epoch 1 Step 224 발산).

- **쌍선형 보간(Bilinear Interpolation)으로 인해 영상이 미세하게 흐려지는 현상 (`RandAffined`):** 임의 각도 회전·크기 변환 시 적용되는 쌍선형 보간은 주변 픽셀의 평균을 구해 빈자리를 채우므로 고주파 잡음이 다소 흐려진 영상을 학습하게 만들고, 보간이 없는 원본 테스트셋(`L192`, `L506`)을 추론할 때 학습 데이터와 테스트 데이터 간 미세한 질감 차이를 유발할 수 있다. 실제로 보간이나 밝기 변형이 없는 직각(90°) 회전(`RandRotate90d`)과 상하·좌우 반전(`RandFlipd`)만 결합한 `Rot90_Flip_Combo`를 적용했을 때 `U_Net`(27.17 dB → 27.34 dB)과 `AttU_Net`(27.24 dB → 27.37 dB)에서 소폭의 성능 개선을 확인할 수 있었다.

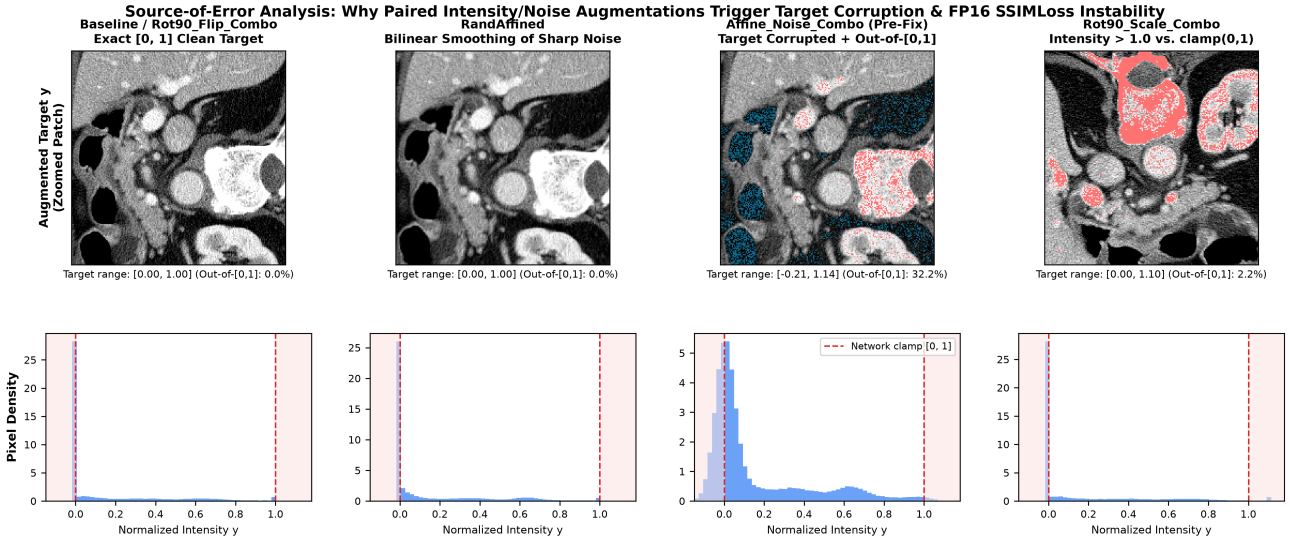


그림 8: 4종 증강 기법 적용 시 정답(full) 패치 및 정규화 픽셀값 분포 변화. 적색/청색 음영은 구간 $[0,1]$을 벗어나 정답과 출력의 범위 불일치 및 FP16 SSIMLoss 수치 불안정성을 유발할 수 있는 픽셀 영역을 나타냄

6.3 오류 원인 3: 슬라이스 단위 무작위 검증 분할과 환자 단위 테스트 분할의 차이

U_Net_none에서 검증 최고 점수(val_psnr $= 25.88$ dB)와 독립 테스트 점수(L192: 26.93 dB, L506: 27.44 dB, 통합 27.17 dB) 간에는 약 1.3 dB의 수치 차이가 관찰된다. 이는 8명 학습·검증 환자군 내에 체구가 크거나 광자 부족(Photon Starvation) 잡음이 더 심한 촬영본이 포함된 영향과 함께, ShuffleSplit이 환자 단위가 아닌 슬라이스 단위로 80:20 무작위 분할을 수행하여 인접한 ± 3mm 슬라이스 간 해부학적 유사성이 검증 점수에 반영되는 한계가 있기 때문으로 생각된다.

7. 결론 및 고찰 (Conclusion)

본 과제에서는 원본 512×512 Raw DICOM (.IMA, 3mm Sharp 커널) 데이터셋과 $[-100, 300]$ HU 전처리 파이프라인을 바탕으로, 입·출력 해상도 유지(padding=1) 및 잔차 연결(+ x)이 적용된 U-Net 계열 복원 모델들을 학습하고 비교하였다. 실험 결과, 잔차 학습과 $L_1 + 0.1 \cdot \mathcal{L}_{\text{SSIM}}$ 복합 손실을 적용한 모델들은 Quarter-Dose CT의 양자 잡음을 완화하면서(PSNR 21.75 dB → 27.37 dB, SSIM 0.7160 → 0.8379, RMSE 33.45 HU → 17.41 HU) 장기 경계와 혈관 구조를 비교적 안정적으로 복원함을 확인하였다. 또한 입·출력 잔차 연결 유무(U_Net vs. U_Net_NoRes) 및 블록 내부 순환·잔차 연결(R2U_Net)을 비교하며 각 구조의 수렴 속도와 연산량 특성을 살펴볼 수 있었다.

끝으로 본 과제를 수행하며 시행착오를 통해 배운 점과 향후 보완이 필요한 한계점은 다음과 같다:

1. **혼합 정밀도 학습 시 수치 안정성 및 출력 범위 처리의 중요성:** 초기 실험에서 출력단의 무조건적인 clamp(0, 1) 적용이 잔차 연결이 없는 모델(U_Net_NoRes)의 초기 기울기 소실을 유발할 수 있다는 점과, FP16 혼합 정밀도 환경에서 SSIMLoss의 지역 분산 계산($G(y^2) - \mu_y^2$)이 수치 발산(NaN)에 취약하다는 점을 시행착오를 통해 확인하였다. 학습 시 선형 출력을 허용하고 손실 함수 계산을 32비트(float32)로 분리하는 간단한 수정만으로도 학습 안정성이 크게 달라진다는 점을 배울 수 있었다.
2. **영상 복원 과제에 맞는 데이터 증강 선택 및 검증 분할의 한계:** 세분화 과제와 달리 저선량 CT 복원에서는 정답 영상에 노이즈나 밝기 스케일링, 쌍선형 보간을 가할 경우 정답 영상 오염과 미세한 흐려짐이 발생함을 확인하였으며, 픽셀값과 잡음 특성을 바꾸지 않는 직각 회전 및 상하·좌우 반전 증강(Rot90_Flip_Combo)이 복원 과제에 더 적합함을 확인하였다. 다만 제한된 GPU 연산 자원으로 인해 순환 연산량이 큰 모델(R2U_Net 등)을 충분한 에포크 동안 학습시키지 못한 한계가 있으며, 향후에는 슬라이스 단위 분할 대신 환자 단위 교차 검증(GroupKFold)과 인접 슬라이스 정보를 함께 입력받는 2.5D/3D 복원 구조를 추가로 탐구해 보고자 한다.